Electronics & Computing Club

Car Proximity Alarm Project

Build & Code: Arduino Workshop

Written by Charles Ross on January 5th, 2026

Section 1
Overview

It seems that sensors are all over cars these days, and for good reason too! If you've ever had the displeasure of parallel parking downtown, you'd know how many times that annoying alarm saved you from a 500$ paint job. For this project, you will learn exactly how to use one of these sensors to work an alarm. A little more barebones than your usual reverse cameras, but it'll get the job done!

The project is going to require two things: a sensor and an alarm system. The sensor will give you a distance value to the object it's looking at, and you will activate the alarm if that object is too close. We have two modules for those purposes: the ultrasonic sensor, which emits echolocation signals to detect how close an object is to it, and the active buzzer which will emit a sound whenever you send a high signal to it.

The project is going to require what we call *libraries*. They, essentially, contain code that someone else has already written so you don't need to start from scratch when making a program. Specifically, we will use a library for the ultrasonic sensor, so you won't be required to create custom code to read distance information.

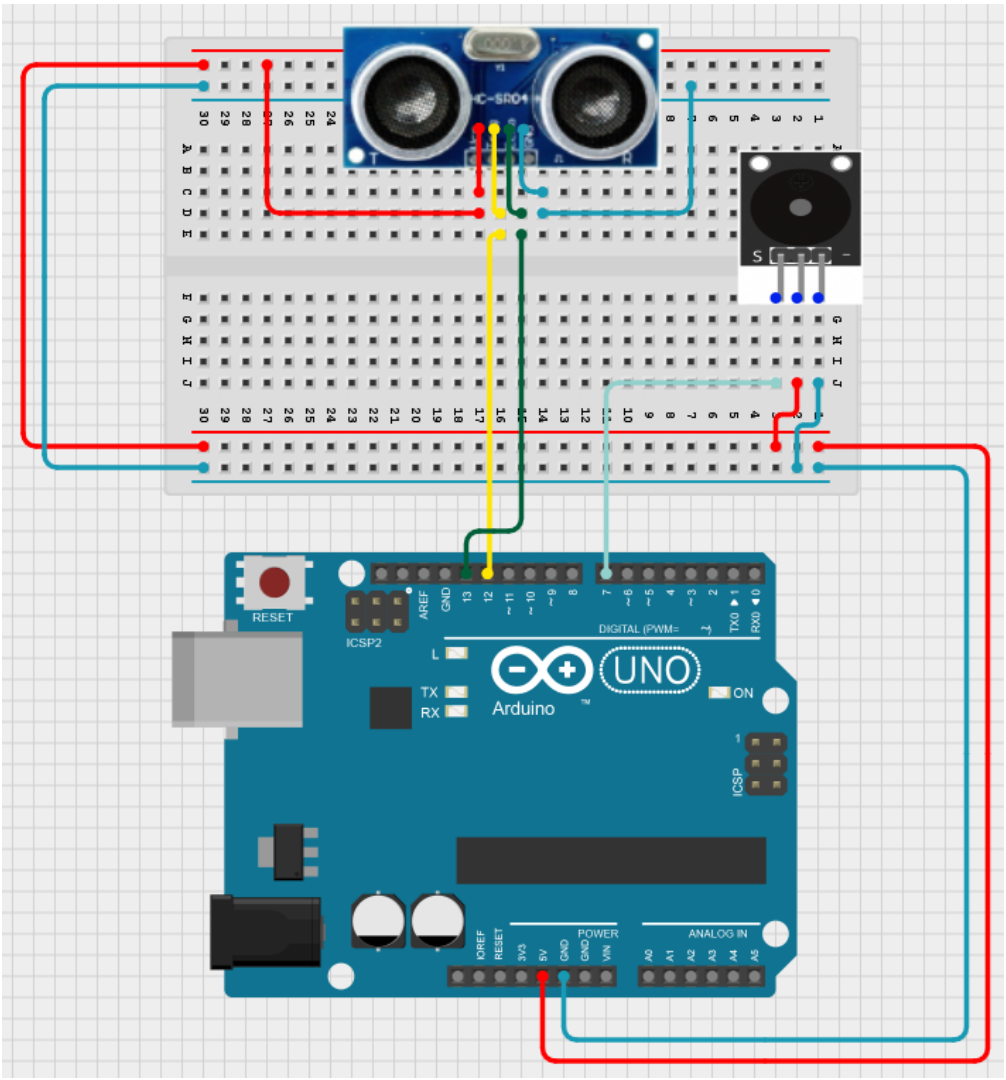
Section 2
Required Components

You will need the following components:

- 1x Ultrasonic Sensor
- 1x Active buzzer module (WARNING: DO NOT REMOVE THE STICKER)
- 1x Breadboard
- 1x Arduino Uno
- Assorted Breadboard Wires
- Assorted Jumper Wires

Section 3

Circuit Assembly



https://app.cirkitdesigner.com/project/7f24109c-5c8e-4c0f-ada5-6a42b7a84ef3

Please refer to the image above for the complete circuit wiring. A link to the online schematic can be found under the image in case you need higher resolution.

If this is your first time using a breadboard (the white rectangle in the image with many holes), please refer to our *Breadboard Guide*.

1. **Start by connecting your 2 power bus lines together**
   The two "+" lines should be connected with a wire and vice-versa for the two "-" lines.

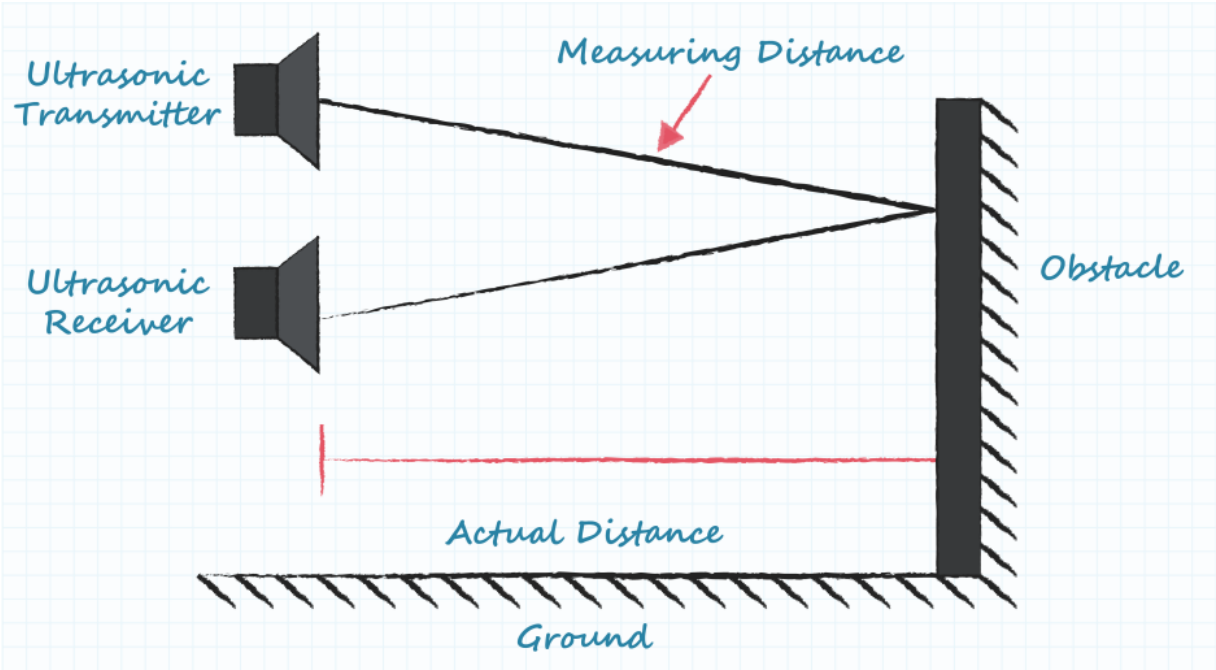
2. **Place your components as indicated by the diagram**
   For the ultrasonic sensor, place it close to the edge of the breadboard and make sure the two speaker components are facing away from the breadboard.

3. **Start by wiring the power connections**
   Make sure the Arduino is not connected to USB before you do this. Connect the Arduino 5V pin to the "+" line and one of the Arduino GND pins to the "-" line. For the ultrasonic sensor, connect the "VCC" labelled pin to the "+" line and the "GND" labelled pin to the "-" line. For the active buzzer, connect the "-" labelled pin to the "-" line and the middle pin to the "+" line.

4. **Wire up your signal connections to the Arduino**
   Connect the Arduino pin 13 to the ECHO labelled pin, and the Arduino pin 12 to the TRIG labelled pin on the ultrasonic sensor. Connect the Arduino pin 7 to the "S" labelled pin on the active buzzer.

The ultrasonic sensor works by sending out a sound wave from one of the speakers, controlled by the TRIG pin. Then, the sound wave will hit an obstacle and echo back to the sensor, which will be picked up by the other speaker, controlled by the ECHO pin. This is why there are two speakers on the sensor: one sends a sound wave and the other receives it. The amount of time it took for the sound to be picked up by the receiving speaker will determine the distance detected by the ultrasonic sensor.

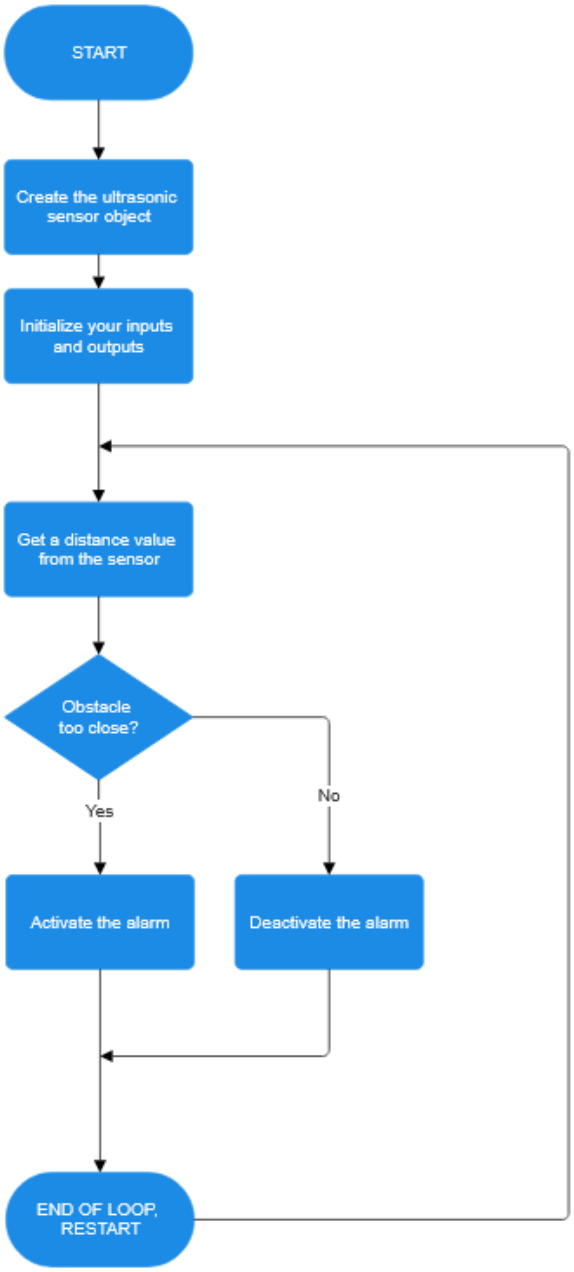
The following diagram illustrates exactly how the ultrasonic sensor works to determine accurate distance measurements.



https://i0.wp.com/edn.com/wp-content/uploads/ep-ultrasonic-sensors-figure-2.png?resize=936%2C514&ssl=1

Section 4
Coding

Use this flowchart to guide your coding:



To use the ultrasonic sensor library, you need to download it first. You can find a small procedure in the *Coding Dictionary* if you don't know how to download it from the library manager. The library you're looking for is *NewPing* by Tim Eckel.

Once that's downloaded, you need to add your *#include* statement. This is what you need to put in the code to tell the compiler that you're using the *NewPing* library in the code. Otherwise, it will not recognize the objects and methods you code.

```
#include <NewPing.h>
```

Then, you need to create an object. The *NewPing* object requires three parameters: the trigger pin, the echo pin, and the max distance in cm. By default, the max distance is 500cm. We don't need that large of a range, so we can reduce the maximum distance to 200cm to make the sensor more reliable. This is to be placed outside of void setup() and void loop().

```
NewPing sonar([trigger pin], [echo pin], [max cm distance])
```

You can initialize your inputs and outputs with the *pinMode()* function. Just to be clear, you don't need to initialize the ultrasonic sensor pins, so it's fine to leave those as-is.

```
pinMode([pin number], {OUTPUT/INPUT})
```

Then, you enter the infinite loop. In this state, you need to actively check distance using the *ping_cm()* method, which can be done like so:

```
distance_value = sonar.ping_cm();
```

One important note: when your distance is greater than the maximum distance, this method will give you a 0. This means that at 0, you shouldn't be activating the alarm.

Then, you enter your decision tree. It's important to understand that the active buzzer would be constantly activating the alarm unless you explicitly deactivate it. Activating and deactivating the alarm is as simple as sending a high or low to the buzzer module's S pin. You can accomplish this with the *digitalWrite()* function.

```
digitalWrite([pin number],{HIGH/LOW})
```

We leave you to figure out the rest. Here is the NewPing documentation if you want to dive deeper into ultrasonic sensors. Make sure to refer to the *Coding Dictionary* for useful functions, libraries and terms.

Section 5

Challenge Yourself

1. Visual Indicator

Even if you can't hook up a camera to this alarm system, it might still be nice to have some visual indicator to be able to warn you not to get too close without an alarm blaring at you. Add the RGB LED module to your circuit, and have it display the color green when the sonar detects a safe distance, yellow when the distance is close to the threshold, and red when the distance is below the threshold. You can do this by sending a digital signal to the R, G and B pins to get the color you want. The following table details which pins you need to light up to get the right color (it might not be very accurate, but this is good enough for a simple visual indicator). Remember to put a current-limiting resistor (about 330Ω) to the R, G and B pins between the Arduino connection and the LED module.

Color	R pin	G pin	B pin
Green	0	1	0
Yellow	1	1	0
Red	1	0	0

2. Coding without the library

As useful as the library is, it's more fun to make your own functions. That said, it'll take some knowledge of how the sensor works, and you'll need some applied math. This time, it'll be important to initialize your echo pin as an input and your trigger pin as an output.

We start with the math part. In order to calculate the distance of your sensor, you will need to know the speed of the sound wave and the amount of time it takes to return to the sensor. This handy formula can help you determine that distance. The speed of sound is 0.0343cm/μs (centimeters per microseconds), which leaves the duration (in microseconds) as our variable. Because the sound wave is travelling back and forth, we divide our total result by 2 to get our total centimeter distance value.

$$d_{cm} = \frac{0.0343 * t_{\mu s}}{2}$$

All we're left with is the operation of the sensor. To make the sensor output a sound wave, you need to make sure the TRIG pin is low for a few microseconds, then high for 10 microseconds, and then low again. The *delayMicroseconds()* and *digitalWrite()* functions can effectively accomplish this task (see *Coding Dictionary*), and you will begin the process of receiving a distance value. The sensor, after generating this sound wave, will soon receive a signal on its ECHO pin.

Using the *pulseIn()* function, you will be able to get your duration value for the wave to travel back and forth. It is important to note that the ECHO pin is high by default, so the *pulseIn()* function must be configured to detect the duration of a HIGH pulse. This is because once the wave has gotten to your receiver, the ECHO pin will go low. This is an example of how you would use the *pulseIn()* function:

```
duration = pulseIn([ECHO pin], HIGH);
```

Finally, once you get your duration reading, it's as simple as plugging it into the formula here and sending it out.

Be sure to ask questions if something doesn't make sense. Also, refer to the *Coding Dictionary* for anything from function syntax to possible solutions for your problems.